

Diamond Chatbot — Data Model & API

Integration Spec

Purpose of this document: Your diamond inventory/business data will continue to live in **your existing software** (your ERP/inventory system). The chatbot will **not store diamond data itself** — instead, every time a customer asks something, the chatbot will call APIs that **you build and expose from your existing software**. This document defines exactly what those APIs need to look like so the chatbot can talk to your system correctly.

This is an **integration spec**, not code. Your developer can implement these APIs in whatever language/framework your existing software already uses (PHP, .NET, Java, Node, etc.) — the chatbot only cares about the API contract (URL, inputs, outputs).

1. Why This Architecture (API-First, Not Data-Upload)

Instead of uploading a CSV/Excel that goes stale, your system stays the **single source of truth**:

- Stock changes the moment you sell a diamond in your existing software → chatbot sees it instantly (live API call, not cached data).
- Price updates happen in your system → chatbot always quotes the current price.
- No duplicate data entry, no sync errors.
- The chatbot acts as a "smart frontend" on top of your existing backend.

Flow: Customer chats → Chatbot decides what's needed → Chatbot calls **your API** → Your API queries your database → Response sent back → Chatbot turns it into a natural language + product-card reply.

2. Diamond Data Model (Fields You Need to Expose)

These are the fields the chatbot needs in order to search, compare, quote, and sell diamonds. Map these to whatever your existing database columns are called.

2.1 Core Identification

Field	Type	Example	Notes
<code>diamond_id</code>	string	"DM-20394"	Unique ID / stock number in your system
<code>certificate_no</code>	string	"GIA-2185847362"	Lab certificate number
<code>lab</code>	string	"GIA"	GIA, IGI, HRD, etc.
<code>certificate_url</code>	string (URL)	link to PDF/image	Optional but valuable for trust

2.2 The 4Cs (Core Diamond Attributes)

Field	Type	Example	Notes
<code>carat</code>	number	1.02	Weight
<code>cut</code>	string	"Excellent"	Excellent / Very Good / Good / Fair / Poor
<code>color</code>	string	"F"	D-Z grading scale
<code>clarity</code>	string	"VS1"	FL, IF, VVS1, VVS2, VS1, VS2, SI1, SI2, I1-I3

2.3 Additional Grading Details

Field	Type	Example
<code>shape</code>	string	"Round", "Princess", "Oval", "Emerald", "Cushion", etc.

Field	Type	Example
polish	string	"Excellent"
symmetry	string	"Very Good"
fluorescence	string	"None" / "Faint" / "Medium" / "Strong"
measurements	string	"6.45 x 6.48 x 3.98 mm"
depth_percent	number	61.5
table_percent	number	57.0

2.4 Commercial Data

Field	Type	Example	Notes
price	number	5200.00	Current selling price
currency	string	"USD"	
discount_percent	number	5	Optional, if applicable
stock_status	string	"In Stock" / "Reserved" / "Sold" / "On Memo"	
quantity_available	number	1	Diamonds are usually unique (qty = 1), but relevant if lab- grown/batch
location	string	"Mumbai Vault"	Warehouse/branch, useful for multi- location dealers

2.5 Media

Field	Type	Example
image_urls	array of strings	["https://.../img1.jpg"]

Field	Type	Example
video_url	string	360° spin video link, if available

2.6 Categorization (helps search/filter)

Field	Type	Example
diamond_type	string	"Natural" / "Lab-Grown"
tags	array of strings	["bridal", "solitaire", "investment-grade"]

3. API Structure — What You Need to Build

All endpoints below should be built in **your existing software** and exposed over HTTPS as REST APIs returning JSON. The chatbot platform will call these using an API key you generate.

3.1 Authentication

Every request from the chatbot to your API must include:

Header	Example	Purpose
Authorization	Bearer <api_key>	Confirms the request is genuinely from the chatbot platform
X-Company-ID	abc-diamonds-001	If your system handles multiple showrooms/branches, this tells you which one

You generate this API key once and enter it into the chatbot platform's admin panel under "Connect Your System."

3.2 Endpoint: Search Diamonds

Used when a customer searches, e.g. *"Show me round diamonds, 1 carat, under \$6000."*

- **Method:** GET
- **Path:** /api/diamonds/search
- **Query Parameters** (all optional, chatbot sends whichever apply):

Parameter	Type	Example
shape	string	round
carat_min	number	0.9
carat_max	number	1.2
color_min	string	F
color_max	string	H
clarity_min	string	VS2
price_min	number	3000
price_max	number	6000
cut	string	Excellent
diamond_type	string	Natural
stock_status	string	In Stock
page	number	1
limit	number	10

Expected Response:

```
{
  "total_results": 24,
  "page": 1,
  "results": [
    {
      "diamond_id": "DM-20394",
      "shape": "Round",
      "carat": 1.02,
      "color": "F",
      "clarity": "VS1",
      "cut": "Excellent",
```

```
    "price": 5200.00,  
    "currency": "USD",  
    "stock_status": "In Stock",  
    "image_urls": ["https://.../img1.jpg"],  
    "certificate_no": "GIA-2185847362"  
  }  
]  
}
```

3.3 Endpoint: Get Diamond Details

Used when a customer clicks/asks about one specific diamond, or when the bot needs full detail for comparison/quotation.

- **Method:** GET
- **Path:** `/api/diamonds/{diamond_id}`

Expected Response: Full object with all fields listed in Section 2 (4Cs, grading details, price, stock, media, certificate).

3.4 Endpoint: Check Stock/Price (Live Check)

Used right before booking/ordering, to reconfirm stock hasn't changed since the search happened (important — diamonds are unique, one-of-a-kind items, so stock must be re-verified before commitment).

- **Method:** GET
- **Path:** `/api/diamonds/{diamond_id}/availability`

Expected Response:

```
{  
  "diamond_id": "DM-20394",  
  "stock_status": "In Stock",  
  "price": 5200.00,  
  "currency": "USD",  
  "last_updated": "2026-08-05T10:15:00Z"  
}
```

3.5 Endpoint: Compare Diamonds

Optional — the chatbot can also do this by simply calling "Get Diamond Details" multiple times. But if you want it optimized in one call:

- **Method:** POST
- **Path:** /api/diamonds/compare
- **Request Body:**

```
{ "diamond_ids": ["DM-20394", "DM-20411", "DM-20502"] }
```

Expected Response: Array of full diamond detail objects (same shape as Section 3.3), so the bot can build a comparison table.

3.6 Endpoint: Hold / Reserve a Diamond

Used when a customer says "Book this one" or "Hold this for me." Diamonds are unique items — this prevents two customers from being sold the same stone at the same time.

- **Method:** POST
- **Path:** /api/diamonds/{diamond_id}/hold
- **Request Body:**

```
{
  "customer_name": "Rahul Mehta",
  "customer_phone": "+91xxxxxxxxx",
  "customer_email": "rahul@example.com",
  "hold_duration_hours": 24,
  "notes": "Requested via chatbot"
}
```

Expected Response:

```
{
  "hold_id": "HOLD-88213",
  "diamond_id": "DM-20394",
  "status": "Held",
  "expires_at": "2026-08-06T10:15:00Z"
}
```

Your system should also **auto-release the hold** if it expires without a confirmed order, and mark the diamond available again.

3.7 Endpoint: Release a Hold

Used if a customer cancels or the hold expires.

- **Method:** POST
- **Path:** /api/diamonds/{diamond_id}/release
- **Request Body:** { "hold_id": "HOLD-88213" }

3.8 Endpoint: Generate Quotation

Used when a customer asks for a formal quote on one or more diamonds.

- **Method:** POST
- **Path:** /api/quotations
- **Request Body:**

```
{
  "customer_name": "Rahul Mehta",
  "customer_phone": "+91xxxxxxxxx",
  "customer_email": "rahul@example.com",
  "diamond_ids": ["DM-20394", "DM-20411"],
  "valid_for_days": 7
}
```

Expected Response:

```
{
  "quotation_id": "QT-55210",
  "pdf_url": "https://yoursystem.com/quotes/QT-55210.pdf",
  "total_amount": 10850.00,
  "currency": "USD",
  "valid_until": "2026-08-12"
}
```

Your system generates the actual PDF (using your existing quotation/invoice format) — the chatbot just links to it and shares it with the customer in chat.

3.9 Endpoint: Place Order

Used when a customer confirms a purchase.

- **Method:** POST
- **Path:** /api/orders
- **Request Body:**

```
{
  "diamond_ids": ["DM-20394"],
  "customer_name": "Rahul Mehta",
  "customer_phone": "+91xxxxxxxxx",
  "customer_email": "rahul@example.com",
  "delivery_address": "...",
  "quotation_id": "QT-55210",
  "payment_status": "Pending"
}
```

Expected Response:

```
{
  "order_id": "ORD-99341",
  "status": "Confirmed",
  "total_amount": 5200.00,
  "currency": "USD",
  "estimated_delivery": "2026-08-15"
}
```

3.10 Endpoint: Get Order Status

Used when a customer asks "Where is my order?"

- **Method:** GET
- **Path:** /api/orders/{order_id}

Expected Response:

```
{
  "order_id": "ORD-99341",
  "status": "Shipped",
  "tracking_number": "IN12345678",
  "estimated_delivery": "2026-08-15"
}
```

3.11 Optional: Webhook for Stock/Price Changes

If you want real-time push updates (instead of the chatbot always pulling), your system can call this **chatbot-side** webhook whenever a diamond's price or stock changes:

- **Chatbot's Webhook URL:** provided to you in the chatbot admin panel (e.g., `https://chatplatform.com/webhooks/{company_id}/inventory-update`)
- **Payload you send:**

```
{
  "diamond_id": "DM-20394",
  "stock_status": "Sold",
  "price": 5200.00,
  "updated_at": "2026-08-05T11:02:00Z"
}
```

This keeps the chatbot's cached search index fresh without needing to call your search API on every single message (improves speed).

4. How the Chatbot Uses These APIs (End-to-End Example)

Customer: "Show me round diamonds, 1 carat, VS1 or better, under \$6000"

→ Chatbot calls `GET /api/diamonds/search?`

`shape=round&carat_min=0.95&carat_max=1.05&clarity_min=VS1&price_max=6000`

→ Your API returns matching diamonds → Chatbot shows them as cards.

Customer: "Compare the first two"

→ Chatbot calls `POST /api/diamonds/compare` with those two IDs → shows comparison table.

Customer: "Book the second one"

→ Chatbot calls `GET /api/diamonds/{id}/availability` to reconfirm it's still in stock

→ Then calls `POST /api/diamonds/{id}/hold` with customer details

→ Confirms to customer: "Held for 24 hours, reference HOLD-88213."

Customer: "Send me a quotation"

→ Chatbot calls `POST /api/quotations` → shares PDF link in chat.

Customer: "I'll take it, place the order"

→ Chatbot calls `POST /api/orders` → confirms order ID and delivery estimate.

Customer (next day): "Where's my order?"

→ Chatbot calls `GET /api/orders/{order_id}` → shows live status.

5. What You Need to Build on Your Side (Checklist)

- ☐ `GET /api/diamonds/search`
 - ☐ `GET /api/diamonds/{id}`
 - ☐ `GET /api/diamonds/{id}/availability`
 - ☐ `POST /api/diamonds/compare` *(optional — can be skipped, chatbot can call details endpoint multiple times instead)*
 - ☐ `POST /api/diamonds/{id}/hold`
 - ☐ `POST /api/diamonds/{id}/release`
 - ☐ `POST /api/quotations`
 - ☐ `POST /api/orders`
 - ☐ `GET /api/orders/{id}`
 - ☐ *(Optional)* Webhook push for stock/price changes
 - ☐ Generate an API key and share it in the chatbot platform's admin panel
 - ☐ Make sure all endpoints are behind HTTPS with the `Authorization` header check
-

6. Notes for Your Developer

- All responses should be in JSON, UTF-8 encoded.
- Use consistent field names across endpoints (e.g., always `diamond_id`, never mix with `id` or `stone_id` in different places).
- `stock_status` values should be a fixed, agreed set: `In Stock`, `Reserved`, `Sold`, `On Memo` — the chatbot logic depends on exact string matching (or you can send numeric status codes if you prefer — just tell the chatbot team so it's mapped correctly).

- Since diamonds are typically **one-of-a-kind items** (quantity = 1), the **hold/reserve** step (3.6) is critical — without it, two customers could be shown the same "available" diamond and both try to buy it.
- Timestamps should be in ISO 8601 format with timezone (e.g., 2026-08-05T10:15:00Z).
- Recommend rate-limiting the search endpoint on your side (e.g., max 60 requests/minute) to protect your database from overload — the chatbot platform will respect standard 429 Too Many Requests responses and retry politely.